

ThermoGuard Vibecoded

Máximo Justiniano, Rodrigo Ramato, Lucía Castiñeiras, Santiago Arab
44834085, 45129672, 44700138, 46500280
Martes Noche, M3

¹Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen.

El presente informe detalla el proceso de desarrollo y el análisis de métricas correspondientes a la Actividad Integradora, enfocándose específicamente en la implementación del ecosistema móvil (Android) diseñado para interactuar con un sistema embebido ESP32. El objetivo principal de este estudio es contrastar el paradigma de ingeniería de software tradicional —desarrollo de código manual— frente al enfoque emergente de "Vibecoding Total", mediante el cual la lógica y arquitectura de la aplicación son generadas íntegramente por modelos de Inteligencia Artificial a partir de directrices en lenguaje natural.

A lo largo de este documento, se documenta la estrategia de ingeniería de prompts utilizada para replicar la aplicación original, asegurando el cumplimiento de todos los requisitos funcionales, como la ejecución en background y la gestión de sensores. Asimismo, se expone una comparativa del impacto de ambos enfoques, evaluando métricas críticas de rendimiento y analizando la eficiencia en los tiempos de desarrollo. Este análisis permite dimensionar la viabilidad y el impacto de integrar herramientas de IA generativa en la creación de soluciones de Internet de las Cosas (IoT).

1 Herramienta utilizada

Gemini

2 Modelo de IA utilizado

3.1 Pro

3 Prompt técnico utilizado

Actúa como un Arquitecto de Software Senior experto en Android e IoT. Tu objetivo es generar el código completo de una aplicación llamada ThermoGuard utilizando Kotlin y Gradle Kotlin DSL. La aplicación debe ser modular, reactiva y capaz de gestionar telemetría térmica en tiempo real.

1. STACK TECNOLÓGICO Y CONFIGURACIÓN

- Lenguaje: Kotlin 2.2+.
- SDK: Min 24, Target 36.
- Librerías:
 - org.eclipse.paho:org.eclipse.paho.client.mqttv3:1.2.5
 - org.eclipse.paho:org.eclipse.paho.android.service:1.1.1
 - com.google.android.material:material:1.14.0
 - androidx.appcompat, constraintlayout, core-ktx.
- Permisos: INTERNET, VIBRATE, POST_NOTIFICATIONS, FOREGROUND_SERVICE, FOREGROUND_SERVICE_SPECIAL_USE (Subtipo: "MQTT connection for real-time security alerts").
- Red: Habilitar android:usesCleartextTraffic="true" en el Manifest.

2. ARQUITECTURA DE RED Y CONSTANTES (Constants.kt)

Centraliza todos los valores en un objeto Constants:

- Tópicos MQTT:
 - grupoM3/sensor/temperatura (Lectura)
 - grupoM3/sensor/estado (Lectura crítica)
 - grupoM3/set/modo (Escritura - Payloads: "BAJO", "MEDIO", "ALTO")
 - grupoM3/comando (Escritura - Payloads: "ENCENDER", "APAGAR")
- API REST: <http://172.20.10.8:1880/api/historial>
- Rangos Térmicos: Frio < 16.6 < Medio < 33.3 < Caliente (Max 50.0).

3. NÚCLEO DE COMUNICACIÓN (MqttManager & MqttService)

- MqttManager (Singleton): Gestiona la conexión asíncrona (Threads) y mantiene una lista de listeners ((String, String) -> Unit) para que múltiples componentes reciban mensajes simultáneamente.

- MqttService (Foreground Service):

- Mantiene la conexión MQTT persistente.

- Escucha el tópico sensor/estado. Si recibe "ALERTA" o "ALARMA" (normalizar a uppercase/trim), dispara una Notificación de Alta Prioridad con canal de alerta, sonido y vibración de 1000ms.

4. MÓDULOS FUNCIONALES (Activities)

- 1.MainActivity (Login): Formulario para configurar Broker/Puerto con persistencia en SharedPreferences. Inicia el MqttService tras conectar.

- 2.ConfigActivity (Dashboard): Panel con MaterialCardView. Actualiza dinámicamente el icono y color del estado del hardware (IDLE = Gris, MONITOREANDO = Verde, ALERTA = Rojo) según los mensajes de sensor/estado.

- 3.ThermometerActivity & ThermometerView:

- Crea una Custom View que dibuje un termómetro con Canvas.

- Anima el mercurio con ValueAnimator y cambia el color según los rangos térmicos definidos.

- Botón para refrescar la última temperatura desde la API REST.

- 4.SensorActivity (Gesto Shake):

- Usa el acelerómetro. Detecta sacudida si $\sqrt{x^2+y^2+z^2} - 9.8 > 13$.

- Al detectar shake, publica "APAGAR" en el tópico comando para desactivar alarmas.

- 5.ModosActivity (Control): Botones para enviar "BAJO", "MEDIO" o "ALTO" al tópico set/modo con QoS 1.

- 6.HistorialActivity (REST API):

- Petición GET asíncrona a la API.

- Importante: Esperar un JSON con estructura {"data": [{"device_id": "...", "temperature": 0.0, "created_at": "..."}]}.

- Mostrar en RecyclerView con un HistorialAdapter.

5. REGLAS DE DISEÑO Y CICLO DE VIDA

- Hilos: Jamás realizar operaciones de red en el Main Thread. Usar runOnUiThread para actualizar la UI.

- Sensores: Registrar el listener en onResume y liberar en onPause.

- Validación: Limpiar siempre los payloads MQTT (remover comillas, espacios y normalizar a mayúsculas).

Genera el código completo siguiendo esta estructura, incluyendo todos los layouts XML, la lógica de los adaptadores y la configuración del Manifest.

5 Cantidad de tokens de IA utilizados

30.000 tokens

6 Tiempo para obtener una versión funcional

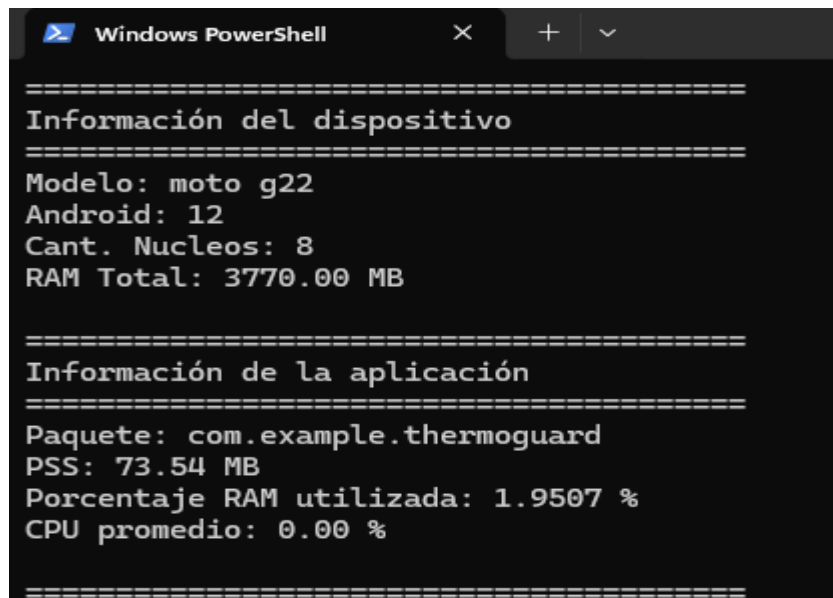
Código manual: 2 semanas

Código con Vibecoding: 2 horas

7 Métricas CPU y Memoria

Código manual:

Realizando acción con el sensor de Android:



```
Windows PowerShell

=====
Información del dispositivo
=====
Modelo: moto g22
Android: 12
Cant. Nucleos: 8
RAM Total: 3770.00 MB

=====
Información de la aplicación
=====
Paquete: com.example.thermoguard
PSS: 73.54 MB
Porcentaje RAM utilizada: 1.9507 %
CPU promedio: 0.00 %
=====
```

Ejecutando 10 segundos sin interacciones:

```
Windows PowerShell

=====
Información del dispositivo
=====
Modelo: moto g22
Android: 12
Cant. Nucleos: 8
RAM Total: 3770.00 MB

=====
Información de la aplicación
=====
Paquete: com.example.thermoguard
PSS: 67.56 MB
Porcentaje RAM utilizada: 1.7919 %
CPU promedio: 0.00 %
=====
```

Codigo vibecoded:

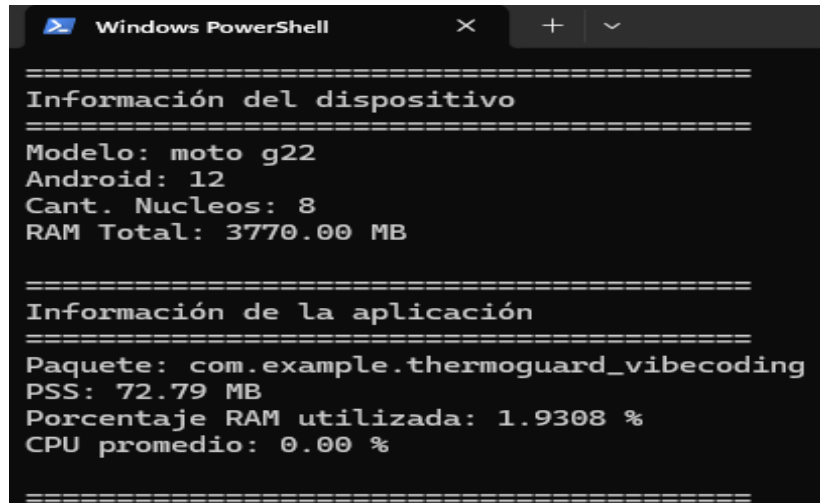
Realizando acción con el sensor de Android:

```
Windows PowerShell

=====
Información del dispositivo
=====
Modelo: moto g22
Android: 12
Cant. Nucleos: 8
RAM Total: 3770.00 MB

=====
Información de la aplicación
=====
Paquete: com.example.thermoguard_vibecoding
PSS: 76.04 MB
Porcentaje RAM utilizada: 2.0170 %
CPU promedio: 0.00 %
=====
```

Ejecutando 10 segundos sin interacciones:



```
Windows PowerShell

=====
Información del dispositivo
=====
Modelo: moto g22
Android: 12
Cant. Nucleos: 8
RAM Total: 3770.00 MB

=====
Información de la aplicación
=====
Paquete: com.example.thermoguard_vibecoding
PSS: 72.79 MB
Porcentaje RAM utilizada: 1.9308 %
CPU promedio: 0.00 %
=====
```

8 Conclusión

La implementación y evaluación de ambas versiones de la aplicación móvil nos permitió evidenciar tanto las ventajas operativas como las limitaciones técnicas del "Vibecoding Total" en el ciclo de vida del software. Si bien la generación de código mediante IA logró reducir drásticamente los tiempos de desarrollo netos, facilitando la implementación rápida de funcionalidades complejas, esta inmediatez conlleva contrapartidas significativas.

Observamos que, en términos de interfaz de usuario, los resultados generados por IA tienden a ser estéticamente genéricos y carecen del refinamiento visual y la coherencia en la experiencia de usuario (UX) que un desarrollador humano prioriza intencionalmente. Asimismo, identificamos que el código producido, aunque funcional, presenta en ocasiones una complejidad innecesaria o una estructura redundante. Al no poseer una visión holística orientada al mantenimiento a largo plazo, la IA tiende a proponer soluciones que, si bien resuelven el problema inmediato, resultan más difíciles de interpretar, depurar y refactorizar manualmente, incrementando potencialmente la deuda técnica en comparación con el código artesanal.

En conclusión, la integración de la IA en el flujo de trabajo requiere un rol de supervisión humana más estricto. La IA actúa como un potente acelerador, pero su producción debe ser rigurosamente curada; el ahorro de tiempo inicial obtenido mediante la generación automática puede transformarse en un mayor esfuerzo de mantenimiento si no se aplican criterios de ingeniería crítica para auditar la calidad estructural del código y optimizar la arquitectura de la interfaz.